CENTRALIZED ELECTRONIC HEALTH RECORD (EHR) SYSTEM

A PROJECT REPORT

Submitted by

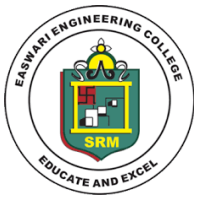
Shanmugapriyan S (310624243172)

Course: Full Stack Development (2311CSC503R)

in partial fulfillment for the award of the Degree of
[DEGREE NAME]
In
[DEPARTMENT NAME]

EASWARI ENGINEERING COLLEGE
(AN AUTONOMOUS INSTITUTION)
AFFILIATED TO ANNA UNIVERSITY, CHENNAI
RAMAPURAM, CHENNAI - 600089

[MONTH-YEAR]



EASWARI ENGINEERING COLLEGE
(AN AUTONOMOUS INSTITUTION)
RAMAPURAM, CHENNAI - 600089

BONAFIDE CERTIFICATE

Certified that this project report titled "Centralized Electronic Health Record (EHR) System" is the Bonafide work of "Shanmugapriyan S (310624243172)", who carried out the project work under my supervision for the course Full Stack Development (2311CSC503R), in partial fulfillment of the requirements for the award of the degree of [DEGREE NAME] in [DEPARTMENT NAME] at Easwari Engineering College, Chennai.

HEAD OF THE DEPARTMENT	SUPERVISOR / PROJECT GUIDE
[HoD Name]	**[Guide Name]**
Head of the Department	**[Designation]**
[Department Name]	**[Department Name]**
Easwari Engineering College	**Easwari Engineering College**
Ramapuram, Chennai - 600089	**Ramapuram, Chennai – 600089**

EASWARI ENGINEERING COLLEGE
(AN AUTONOMOUS INSTITUTION)
CHENNAI - 600089

Certificate of Viva-voce Examination

This is to certify that the Viva-Voce Examination of the Project Work entitled "Centralized Electronic Health Record (EHR) System" submitted by Shanmugapriyan S (Reg. No. 310624243172) in partial fulfillment of the requirements for the award of the degree of [DEGREE NAME] was conducted on _____ (Date) in the Department of [DEPARTMENT NAME], Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner	**External Examiner**
Date:	**Date:**

ABSTRACT

The Centralized Electronic Health Record (EHR) System is a web-based healthcare management platform that provides administrators, doctors, and patients with a unified environment to store, retrieve, and manage clinical and administrative health information, directly resolving the inefficiencies of fragmented paper records and siloed departmental databases that lead to data duplication, clinical inconsistencies, and care delays. The system is architected on the modern MERN technology stack: MongoDB, Express.js, React.js, and Node.js. The React front end (built with Vite) provides role-tailored user interfaces, authenticates users via JSON Web Tokens (JWT), and interacts with a modular RESTful Express API. The backend server verifies cryptographic bearer tokens, enforces strict Role-Based Access Control (RBAC) using protect and requireRole middleware, and interfaces with a local MongoDB Community Server database instance through Mongoose schemas. Eight core collections manage system users, patient profiles, doctor profiles, clinical appointments, medical records, digital prescriptions, diagnostic report metadata, and system audit logs. All passwords are encrypted as salted bcrypt hashes, and an automated audit logging service captures authentication events and data-modifying operations. Administrator, doctor, and patient dashboards share a cohesive responsive layout and synchronize live data with the backend API, utilizing mock data only as a resilient fallback mechanism if network communication fails. In the current release, read endpoints are implemented for all entities and patients can create appointments via the REST API; creating medical records, issuing prescriptions, and recording report metadata from the front-end modals update client-side state only and are not yet persisted to the database, and report binary file upload is not implemented. Automated backend test suites and the frontend production build completed with zero errors. The system serves as an academic prototype: it has not undergone clinical validation, formal security auditing, or regulatory compliance certification. Future work includes persisting all client-side write actions, implementing multi-part file handling for diagnostic reports, rate limiting, input validation, pagination, broader automated testing, and production deployment preparation.

ACKNOWLEDGEMENT

I express my sincere gratitude to the Principal, Dr. P. Deiva Sundari, and the Management of Easwari Engineering College (Autonomous) for providing the facilities, academic environment, and opportunity to carry out this project work.

I convey my heartfelt thanks to the Head of the Department, [HoD Name], Department of [Department Name], for the continuous encouragement and guidance provided throughout this course.

I am extremely thankful to my Project Guide, [Guide Name], [Designation], Department of [Department Name], for the constant mentorship, valuable suggestions, and motivation from conceptualization to the completion of this project.

I also thank the faculty and laboratory staff of the department for their assistance, and my parents and friends for their encouragement and support throughout my studies.

Shanmugapriyan S

(Reg. No. 310624243172)

TABLE OF CONTENTS

Chapter Title	Page
Abstract	iv
Acknowledgement	v
List of Tables	viii
List of Figures	ix
1. INTRODUCTION	1
1.1 OVERVIEW	1
1.2 PROBLEM DEFINITION	3
1.3 OBJECTIVES OF THE PROJECT	4
1.4 SCOPE OF THE PROJECT	5
1.5 MOTIVATION	6
2. LITERATURE SURVEY	8
2.1 LITERATURE REVIEW	8
2.2 EXISTING SYSTEMS AND LIMITATIONS	10
2.3 PROPOSED EHR SYSTEM	11
3. SYSTEM ANALYSIS & SPECIFICATION	13
3.1 SYSTEM REQUIREMENTS ANALYSIS	13
3.1.1 Functional Requirements	13
3.1.2 Non-Functional Requirements	15
3.2 SOFTWARE AND HARDWARE REQUIREMENTS	16
3.2.1 Software Specifications	16
3.2.2 Hardware Specifications	17
4. SYSTEM DESIGN AND METHODOLOGY	18
4.1 MERN SYSTEM ARCHITECTURE	18
4.2 MODULAR DECOMPOSITION	21
4.2.1 Authentication Module	22
4.2.2 Role-Based Access Control Module	22
4.2.3 User Management Module	24
4.2.4 Patient and Doctor Management Modules	24
4.2.5 Appointment Management Module	24
4.2.6 Medical Record Management Module	25
4.2.7 Prescription Management Module	25
4.2.8 Report Management Module	25

4.2.9 Audit Logging Module 25
4.2.10 Shared Front-End Modules 26
4.3 USE-CASE AND DATA-FLOW MODELS 26
4.4 ENTITY-RELATIONSHIP AND COLLECTIONS SCHEMA 29
4.5 AUTHENTICATION AND AUTHORIZATION SEQUENCES 31
4.6 APPOINTMENT MANAGEMENT WORKFLOW 33
4.7 REST API DESIGN 35
5. RESULTS AND DISCUSSION 37
5.1 USER INTERFACE IMPLEMENTATION & VERIFICATION 37
5.1.1 Login Page 38
5.1.2 Administrator Dashboard 39
5.1.3 Doctor Dashboard 42
5.1.4 Patient Dashboard 44
5.2 KEY IMPLEMENTATION DETAILS (CODE EXPLANATION) 46
5.2.1 MongoDB Connection (backend/config/db.js) 46
5.2.2 Password Hashing (backend/models/User.js) 47
5.2.3 JWT Generation and Verification 49
5.2.4 Role-Based Authorization Middleware 50
5.2.5 Front-End API Service and Route Guards 51
5.2.6 Audit Log Service 53
5.3 TESTING AND VALIDATION 54
5.3.1 Testing Methodology 54
5.3.2 Untested Areas 55
5.4 IMPLEMENTATION STATUS 55
5.5 SECURITY DISCUSSION 57
5.6 DISCUSSION OF RESULTS 58
6. CONCLUSION AND FUTURE ENHANCEMENTS 59
6.1 CONCLUSION 59
6.2 FUTURE ENHANCEMENTS 60
APPENDIX 63
A.1 PROJECT DIRECTORY STRUCTURE 63
A.2 ENVIRONMENT CONFIGURATION 64
A.3 SAMPLE REST API REQUESTS AND RESPONSES 64
A.4 SEED SCRIPTS 65
REFERENCES 66

LIST OF TABLES

Table No.	Table Description	Page
Table 2.1	Comparison of Existing Practice and the Proposed System	10
Table 3.1	Functional Requirements and Implementation Status	13
Table 3.2	Non-Functional Requirements	15
Table 3.3	Software Requirements and Environment Specifications	16
Table 3.4	Hardware Configuration (to be completed)	17
Table 4.1	Module Summary	21
Table 4.2	Role-Based Access Control Matrix (Verified Routes)	23
Table 4.3	Database Collections and Fields	29
Table 4.4	REST API Endpoint Summary	35
Table 5.1	Testing Summary	54
Table 5.2	Feature Implementation Status	55
Table 5.3	Security Measures: Implemented and Not Implemented	57
Table A.1	Backend Environment Variables	64
Table A.2	Seed and Setup Scripts	65

LIST OF FIGURES

Figure No.	Figure Description	Page
Figure 4.1	Three-Tier MERN Architecture of the EHR System	20
Figure 4.2	Use-Case Diagram of the EHR System	26
Figure 4.3	Data-Flow Diagram of the EHR System (Level 1)	27
Figure 4.4	Entity-Relationship Diagram of the EHR Collections	29
Figure 4.5	Authentication and Authorization Sequence	31
Figure 4.6	Appointment Management Workflow: Backend Path and Current Front-End Behaviour	33
Figure 5.1	Login Page	38
Figure 5.2	Administrator Dashboard Overview	39
Figure 5.3	Administrator – Users Tab	40
Figure 5.4	Administrator – Patient Management	40
Figure 5.5	Administrator – Doctor Management	41
Figure 5.6	Administrator – Audit Logs	42
Figure 5.7	Doctor – Appointment Management	43
Figure 5.8	Doctor – Medical Records	43
Figure 5.9	Doctor – Prescriptions	44
Figure 5.1	Login Page	38
Figure 5.1	Login Page	38
Figure 5.1	Login Page	38

CHAPTER 1
INTRODUCTION

This chapter introduces the background, problem domain, objectives, scope, and motivation of the Centralized Electronic Health Record (EHR) System. It explains why a single web-based platform for health information is useful, and it states clearly which parts of the system are implemented in the current version and which are partial or planned.

1.1 OVERVIEW 1

An Electronic Health Record (EHR) is a digital collection of a patient's health information, such as demographic details, consultations, diagnoses, prescriptions, and diagnostic reports. In many hospitals and clinics this information is still scattered across paper files and separate departmental databases for registration, appointments, clinical notes, prescriptions, and laboratory reports. Such fragmentation causes duplication, inconsistency, and delay in patient care.

The Centralized Electronic Health Record (EHR) System is a full-stack web application built with the MERN technology stack: MongoDB, Express.js, React.js, and Node.js. It provides a single platform where administrators, doctors, and patients can store, retrieve, and manage health-related information according to their roles. The system keeps users, patient profiles, doctor profiles, appointments, medical records, prescriptions, report metadata, and audit logs in one MongoDB database that is accessed through a RESTful Express API.

At run time, the React front end authenticates a user with a JSON Web Token (JWT) and then calls the Express APIs. The backend validates the token, enforces role-based permissions, and performs database operations through Mongoose on a MongoDB Community Server instance. Each of the three roles is presented with its own dashboard so that only the functions relevant to that role are shown.

This report documents the design and implementation of the system as it exists today. Wherever a feature is only

partly connected to the backend, or uses mock data in the user interface, the report says so explicitly.

1.2 PROBLEM DEFINITION 3

Traditional record-keeping in healthcare settings suffers from the following problems:

- **Paper-based and disconnected systems:** Records are kept in several locations, which leads to loss or inconsistency of patient information.
- **Manual data entry and retrieval:** Writing, copying, and searching records by hand is time-consuming and error-prone.
- **Lack of access control:** Without authentication and role separation, anyone with access to a record store may view or modify data.
- **No audit trail:** It is difficult to determine who performed which action on which record.
- **Scattered user interfaces:** Separately built portals for different user groups produce an inconsistent user experience.

The main objectives of the project are:

- **MERN architecture:** To build the system using MongoDB, Express.js, React.js, and Node.js with a clear separation between presentation, application logic, and data persistence.
- **Central data store:** To keep all entities (users, patients, doctors, appointments, medical records, prescriptions, reports, and audit logs) in one MongoDB database reachable through a unified REST API.
- **Secure authentication:** To authenticate users with JWT and to store passwords only as bcrypt hashes.
- **Role-Based Access Control (RBAC):** To restrict API routes and front-end pages to the roles admin, doctor, and patient.
- **Role-specific dashboards:** To provide separate administrator, doctor, and patient dashboards inside a shared layout.
- **Audit logging:** To record authentication events and data-modifying actions in an audit log that administrators can view.

- **Honest status reporting:** To document precisely which features are persisted in the database and which are still client-side or planned.

1.4 SCOPE OF THE PROJECT 5

The scope of the current version covers the following, with the implementation status recorded in Chapter 5:

- User registration and login with JWT, and retrieval of the current user profile.
- Server-side and client-side role-based route protection for the administrator, doctor, and patient roles.
- Read access, through REST endpoints, to users, patients, doctors, appointments, medical records, prescriptions, reports, and audit logs, subject to role permissions.
- An API endpoint for creating an appointment as a patient.
- Administrator, doctor, and patient dashboards that fetch data from the backend.
- Audit logging of authentication events and data-modifying operations.

The following are outside the scope of the current version: persistence of medical-record, prescription, report, and appointment-booking actions performed from the user interface; file upload for reports; email verification and password reset; pagination and advanced search; rate limiting; and production deployment. These are discussed as limitations and future enhancements in Chapter 6.

1.5 MOTIVATION 6

Hospitals and clinics often maintain separate systems for registration, appointments, clinical notes, prescriptions, and laboratory reports. A unified record system improves data consistency, supports faster decision-making, and makes it possible to audit who accessed or changed information. Building such a system also gives practical experience in full-stack engineering: designing a REST API, securing it with token-based authentication and role checks, modelling related data in MongoDB, and building a role-aware React interface.

CHAPTER 2
LITERATURE SURVEY

This chapter reviews the concepts and published work that underpin the design of the EHR system, examines the limitations of conventional record-keeping, and positions the proposed system. The sources are used to justify design decisions; they do not imply that the project conforms to any standard or regulation.

2.1 LITERATURE REVIEW **8**

Electronic health records. Häyrinen, Saranto, and Nykänen [1] reviewed the research literature on the definition, structure, content, use, and impacts of electronic health records and showed that the term is used with varying meanings across studies. The Institute of Medicine [2] described the key capabilities of an EHR system, including health information and data, results management, order entry, communication, patient support, administrative processes, and reporting. These capabilities motivate the

choice of modules in this project: appointments, medical records, prescriptions, and reports are all stored centrally.

Access control. Role-Based Access Control associates permissions with roles rather than with individual users. Ferraiolo and Kuhn [3] introduced role-based access controls as an alternative to purely discretionary and mandatory models, and Sandhu et al. [4] formalised a family of RBAC reference models. The EHR system follows this idea in a simple form: every user holds one role (admin, doctor, or patient), and each protected API route lists the roles that may call it.

Token-based authentication. A JSON Web Token is a compact, signed way of representing claims between two parties, standardised in RFC 7519 [5]. In this project the server issues a signed JWT after a successful login, and the client sends it in the Authorization header of later requests, so the server does not need to keep a session store.

Password storage. Provos and Mazières [6] proposed the bcrypt adaptive hashing scheme, in which a salt and a configurable cost factor make brute-force attacks more

expensive. The project stores passwords only as bcrypt hashes, created by a pre-save hook in the User model.

Web application security. The OWASP Top 10 [7] lists broken access control among the most serious categories of web application risk. This underlines why every protected route in the system passes through token verification and role checking, and why gaps such as missing rate limiting are recorded as limitations in this report.

Architecture and technologies. Fielding's dissertation [8] defines the REST architectural style on which the project's API is modelled. The implementation relies on the official documentation of MongoDB [9], Express [10], Mongoose [11], React [12], Node.js [13], and Vite [14].

2.2 EXISTING SYSTEMS AND LIMITATIONS 10

Conventional record management in many small and medium healthcare settings combines paper files with separate departmental databases. Their main limitations, as they relate to this project, are summarised in Table 2.1.

Table 2.1 Comparison of Existing Practice and the Proposed System 10

Aspect	**Paper / Disconnected Systems**	**Proposed EHR System**
Data location	Multiple locations; records can be lost or become inconsistent	Single MongoDB database accessed through one REST API
Data entry and retrieval	Manual, slow, and error-prone	Web forms and tables that call backend endpoints
Access control	Anyone with access to the files or database can view or change data	JWT authentication and role checks (admin, doctor, patient) on protected routes
Traceability	Hard to tell who did what	Audit log entries created for authentication and data-modifying actions
User interface	Separate, inconsistent portals	Shared dashboard layout with role-specific pages

2.3 PROPOSED EHR SYSTEM 11

The proposed Centralized EHR System addresses these limitations by bringing the main record-keeping functions into one MERN application. Its principal features are:

- **Central database:** All entities are stored in MongoDB and accessed through a unified API.
- **Web interface:** React forms and tables call backend endpoints instead of relying on manual transcription.
- **JWT and RBAC:** The middleware files auth.middleware.js and role.middleware.js validate tokens and restrict routes to specific roles.

- **Audit logging:** A dedicated service, createAuditLog, persists an audit entry for authentication and data-modifying actions.
- **Shared layout:** DashboardLayout.jsx provides a common navigation shell inside which role-specific pages are rendered.

Some user-interface actions of the current version, namely creating medical records, prescriptions, and report entries and booking or cancelling appointments, update only client-side state. This is stated again in Chapters 4 and 5 so that the reader is not given the impression that these actions are already persisted.

CHAPTER 3
SYSTEM ANALYSIS & SPECIFICATION

This chapter specifies the functional and non-functional requirements of the EHR system and lists the software environment used for its development.

3.1 SYSTEM REQUIREMENTS ANALYSIS **13**

3.1.1 Functional Requirements **13**

Table 3.1Functional Requirements and Implementation Status 13

Table 3.1 Functional Requirements and Implementation Status 13

ID	Requirement	Roles	Status
FR1	Register and log in; receive a JWT on success (POST /api/auth/register, POST /api/auth/login).	Public	Implemented
FR2	Retrieve the current user's profile (GET /api/auth/me).	Any logged-in user	Implemented
FR3	Enforce role-based access on the server (protect and requireRole) and on the client (ProtectedRoute).	All	Implemented
FR4	List all system users (GET /api/users).	Admin	Implemented
FR5	List patient profiles (GET /api/patients).	Admin, Doctor	Implemented
FR6	List doctor profiles (GET /api/doctors).	Admin, Patient	Implemented

ID	Requirement	Roles	Status
FR7	List appointments, filtered by role (GET /api/appointments).	Admin, Doctor, Patient	Implemented
FR8	Create an appointment (POST /api/appointments). The booking form in the interface does not yet call this endpoint.	Patient	Partial
FR9	Update appointment status. The interface changes status in client state only.	Doctor	Partial
FR10	List medical records (GET /api/medical-records). Creation from the interface is client-side only.	Doctor, Patient	Partial
FR11	List prescriptions (GET /api/prescriptions). Creation from the interface is client-side only.	Doctor, Patient	Partial
FR12	List report metadata (GET /api/reports). No file upload or download.	Doctor, Patient	Partial
FR13	Record audit entries for authentication and data-modifying actions; list them (GET /api/audit-logs).	Admin	Implemented
FR14	Present role-specific dashboards that fetch data from the backend, with mock fallback data when an API call fails.	All	Implemented
FR15	Create demonstration accounts using seed scripts.	Developer	Implemented

3.1.2 Non-Functional Requirements 15

Table 3.2Non-Functional Requirements

15

Table 3.2 Non-Functional Requirements 15

Requirement	Description	Current Status
Security	Passwords stored as bcrypt hashes; JWT-protected routes; CORS limited to the configured client origin.	Implemented. Rate limiting, HTTPS enforcement, email verification, and password reset are not implemented.

Requirement	Description	Current Status
Maintainability	Separate routes, controllers, models, middleware, and services on the backend; pages, layout, and a service module on the front end.	Implemented
Usability	Role-specific dashboards in a shared layout with a responsive, glass-morphism style.	Implemented. A formal accessibility audit has not been done.
Reliability of responses	Uniform JSON responses of the form { success, message, data } and centralised error handling.	Implemented
Scalability	Handling of large data volumes.	Limited: GET endpoints return full collections; pagination is not implemented.
Auditability	Traceable record of authentication and data-modifying actions.	Implemented at a basic level (no before/after snapshots).

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

3.2.1 Software Specifications 16

Table 3.3Software Requirements and Environment Specifications 16

Table 3.3 Software Requirements and Environment Specifications 16

Layer	Technology (version)	Purpose
Frontend	react 19.2.8, react-dom 19.2.8	UI library and DOM rendering
Frontend	react-router-dom 7.18.4	Client-side routing
Frontend	lucide-react 1.49.0	Icon set
Frontend build	vite 8.3.0, @vitejs/plugin-react 6.1.1	Development server, bundling, hot-module replacement

Layer	Technology (version)	Purpose
Frontend lint	oxlint 1.81.0 (dev)	Linting
Backend	express 4.19.2, cors 2.8.5	HTTP server and cross-origin support
ODM	mongoose 8.3.1	MongoDB object modelling and schema enforcement
Configuration	dotenv 16.4.5	Environment variable loading
Security	jsonwebtoken 9.0.2, bcryptjs 2.4.3	JWT creation and verification; password hashing
Database	MongoDB Community Server (local)	Primary data store; connection string kept in backend/.env
Dev / test	nodemon 3.1.0, mongodb-memory-server 9.2.0	Auto-restart during development; in-memory database for tests

3.2.2 Hardware Specifications 17

The source material for this project does not record hardware measurements or deployment sizing. Table 3.4 therefore leaves the entries to be completed with the actual development machine used.

Table 3.4 Hardware Configuration (to be completed) 17

Component	Development machine used
Processor	[Fill in: processor model]
System RAM	[Fill in: installed RAM]
Storage	[Fill in: free disk space]
Operating system	[Fill in: operating system and version]
Network	[Fill in: connection used]

CHAPTER 4
SYSTEM DESIGN AND METHODOLOGY

This chapter describes the architecture, module decomposition, use cases, data flow, database design, authentication and authorization flow, appointment workflow, and REST API design of the EHR system.

4.1 MERN SYSTEM ARCHITECTURE **18**

The EHR system uses a decoupled client–server architecture with three tiers, shown in Figure 4.1.

- **Presentation tier (React with Vite):** A single-page application. App.jsx defines the routes and the ProtectedRoute and PublicRoute guards. The services/api.js module wraps fetch, attaches the JWT, and stores the token and user in the browser's localStorage.
- **Application tier (Node.js with Express):** server.js loads the environment, connects to the database, and

starts Express on the configured port (default 5000). app.js configures CORS, the JSON body parser, the route groups under /api, and the not-found and error handlers. Requests pass through protect and requireRole before reaching a controller.

- **Persistence tier (MongoDB with Mongoose):** MongoDB Community Server stores eight collections. Mongoose schemas define fields, references, and the pre-save password-hashing hook.

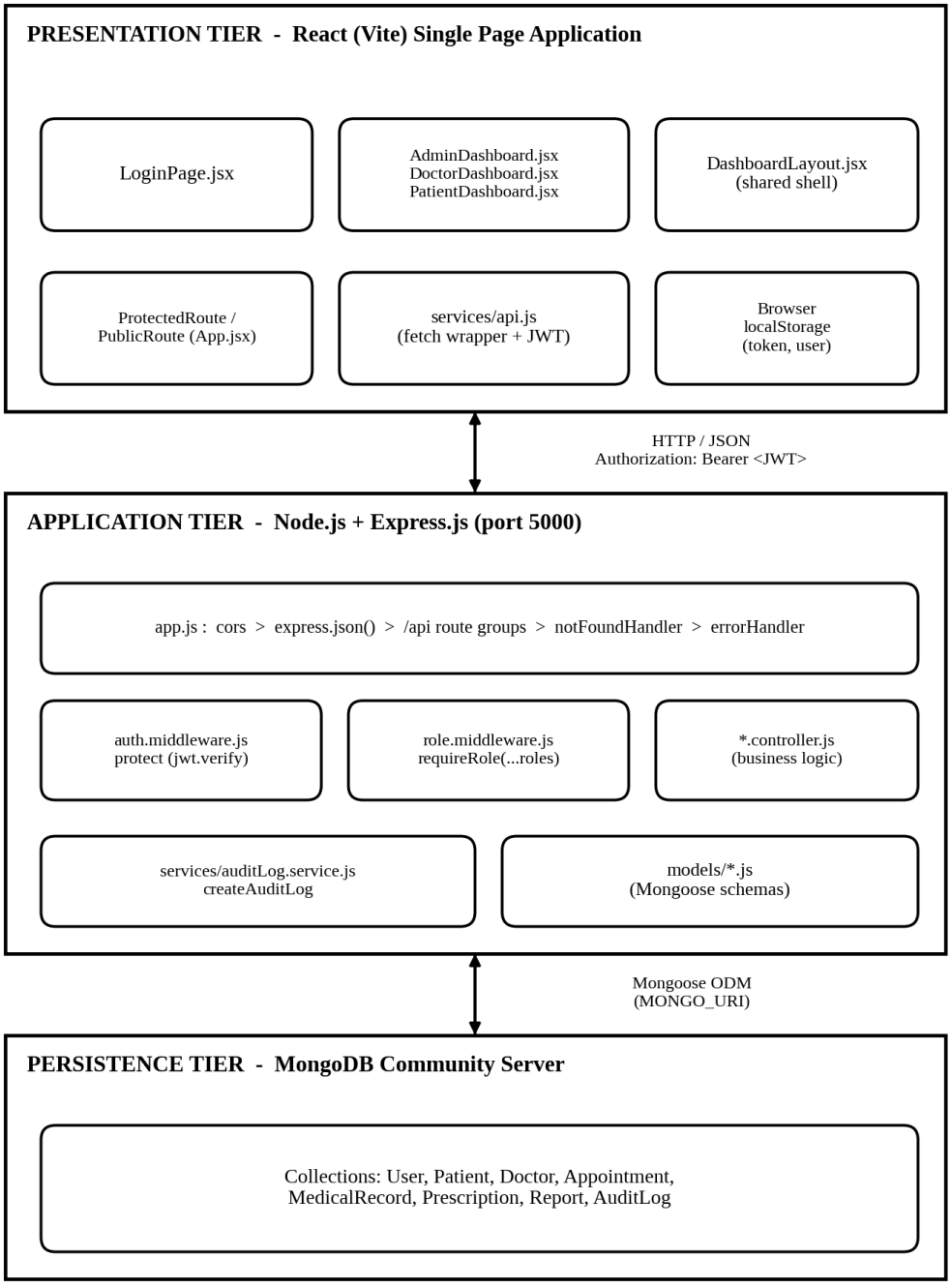


Figure 4.1 Three-Tier MERN Architecture of the EHR System 20

4.2 MODULAR DECOMPOSITION 21

The system is organised into the modules summarised in Table 4.1. The sub-sections that follow describe each module.

Table 4.1 Module Summary 21

Module	**Front-end file(s)**	**Backend route(s)**	**Allowed roles**	**Status**
Authentication	LoginPage.jsx	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me	Public; /me any logged-in user	Implemented
User management	AdminDashboard.jsx (Users tab)	GET /api/users	Admin	Implemented (read)
Patient management	AdminDashboard.jsx (Patients tab)	GET /api/patients	Admin, Doctor	Implemented (read)
Doctor management	AdminDashboard.jsx (Doctors tab)	GET /api/doctors	Admin, Patient	Implemented (read)
Appointment management	DoctorDashboard.jsx, PatientDashboard.jsx	GET /api/appointments, POST /api/appointments	All roles (POST: patient)	Partial
Medical records	DoctorDashboard.jsx (Records tab)	GET /api/medical-records	Doctor, Patient	Partial
Prescriptions	DoctorDashboard.jsx (Prescriptions tab)	GET /api/prescriptions	Doctor, Patient	Partial
Reports	DoctorDashboard.jsx, PatientDashboard.jsx	GET /api/reports	Doctor, Patient	Partial
Audit log	AdminDashboard.jsx (audit view)	GET /api/audit-logs	Admin	Implemented
RBAC middleware	ProtectedRoute in App.jsx	auth.middleware.js and role.middleware.js on protected routes	Per route	Implemented

4.2.1 Authentication Module 22

The authentication module registers and logs in users and issues JWTs. auth.controller.js provides register, login, and getMe. During login the user is found by email with the password field explicitly selected, the supplied password is compared with the stored hash, and generateToken(id, role, email) signs a token using the JWT_SECRET environment variable and an expiry taken from JWT_EXPIRE (default 30 days). Successful registration and login create audit-log entries.

4.2.2 Role-Based Access Control Module 22

RBAC is enforced by two middleware functions. protect extracts the Bearer token from the Authorization header, verifies it with jwt.verify, and attaches the decoded payload to req.user. requireRole(...roles) then compares req.user.role with the roles allowed for the route and returns HTTP 403 if there is no match. On the client, ProtectedRoute checks for a stored token and user, redirects unauthenticated visitors to /login, and redirects a logged-in user who opens another role's dashboard to their own dashboard. The effective permissions are shown in Table 4.2.

Table 4.2 Role-Based Access Control Matrix (Verified Routes) 23

Route	Admin	Doctor	Patient	Unauthenticated
GET /api/health	Yes	Yes	Yes	Yes
POST /api/auth/register, /login	Yes	Yes	Yes	Yes
GET /api/auth/me	Yes	Yes	Yes	No (401)
GET /api/users	Yes	No (403)	No (403)	No (401)
GET /api/patients	Yes	Yes	No (403)	No (401)
GET /api/doctors	Yes	No (403)	Yes	No (401)
GET /api/appointments	Yes	Yes	Yes	No (401)
POST /api/appointments	No (403)	No (403)	Yes	No (401)
GET /api/medical-records	No (403)	Yes	Yes	No (401)
GET /api/prescriptions	No (403)	Yes	Yes	No (401)
GET /api/reports	No (403)	Yes	Yes	No (401)
GET /api/audit-logs	Yes	No (403)	No (403)	No (401)

The matrix reflects the allowed roles recorded for each route. For example, the administrator role is not listed for the medical-record, prescription, and report endpoints, and the doctor role is not listed for the doctor-list endpoint.

4.2.3 User Management Module 24

This module lets an administrator list all users through GET /api/users. The administrator dashboard shows the result in a table on its Users tab.

4.2.4 Patient and Doctor Management Modules 24

Patient profiles (Patient model) and doctor profiles (Doctor model) each reference a User document. GET /api/patients is available to administrators and doctors; GET /api/doctors is available to administrators and patients so that a patient can see doctors when booking an appointment.

4.2.5 Appointment Management Module 24

GET /api/appointments returns appointments filtered by role for any logged-in user, and POST /api/appointments allows a patient to create an appointment by supplying doctorId, appointmentDate, appointmentTime, and reason. The doctor and patient dashboards list appointments fetched from the backend. However, the booking form, the cancel action, and the doctor's status buttons currently change only React state; they do not call the API. The workflow is shown in Figure 4.6.

4.2.6 Medical Record Management Module 25

GET /api/medical-records lists records for doctors and patients. The doctor dashboard provides a modal form for

adding a record, but the new record is added to client state only and is not persisted.

4.2.7 Prescription Management Module 25

GET /api/prescriptions lists prescriptions for doctors and patients. Each prescription stores a medications array whose items hold name, dosage, frequency, and duration. The doctor dashboard modal for issuing a prescription updates client state only.

4.2.8 Report Management Module 25

GET /api/reports lists report metadata (type, name, and date). No file is stored, and upload and download in the interface are mocked.

4.2.9 Audit Logging Module 25

The createAuditLog function in services/auditLog.service.js inserts an AuditLog document that records the user, user role, action, resource, resource identifier, description, and IP address. Authentication and data-modifying controllers call it, and administrators read the entries through GET /api/audit-logs.

4.2.10 Shared Front-End Modules **26**

DashboardLayout.jsx provides the sidebar, top bar, and logout button shared by all roles. services/api.js offers get, post, put, and delete helpers that add the Authorization header automatically and return an object with success, data, message, and status. src/data/mockData.js holds static arrays that are used as fallback when an API call fails.

4.3 USE-CASE AND DATA-FLOW MODELS **26**

Figure 4.2Use-Case Diagram of the EHR System 26

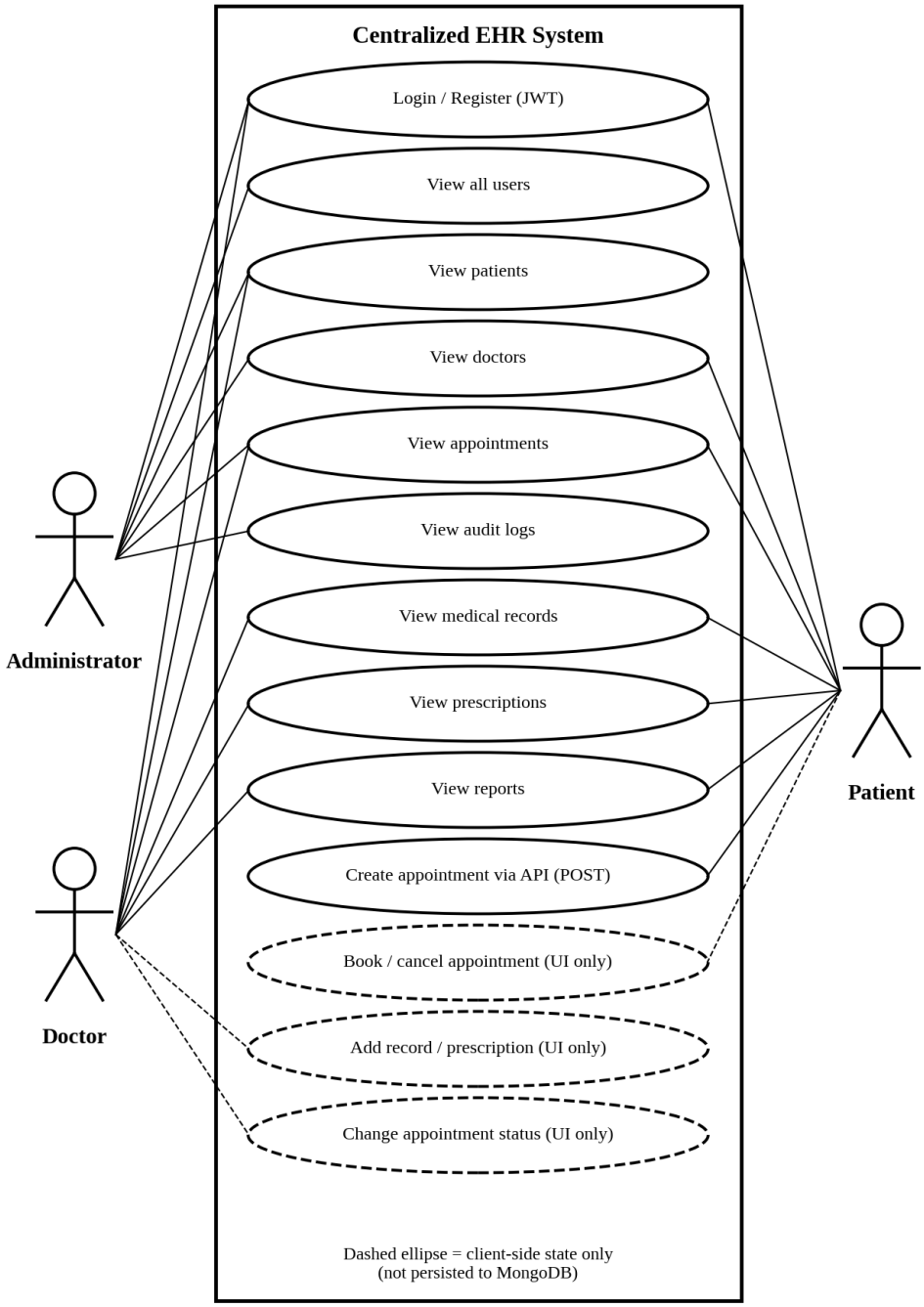


Figure 4.2 Use-Case Diagram of the EHR System 26

Figure 4.3Data-Flow Diagram of the EHR System (Level 1) 27

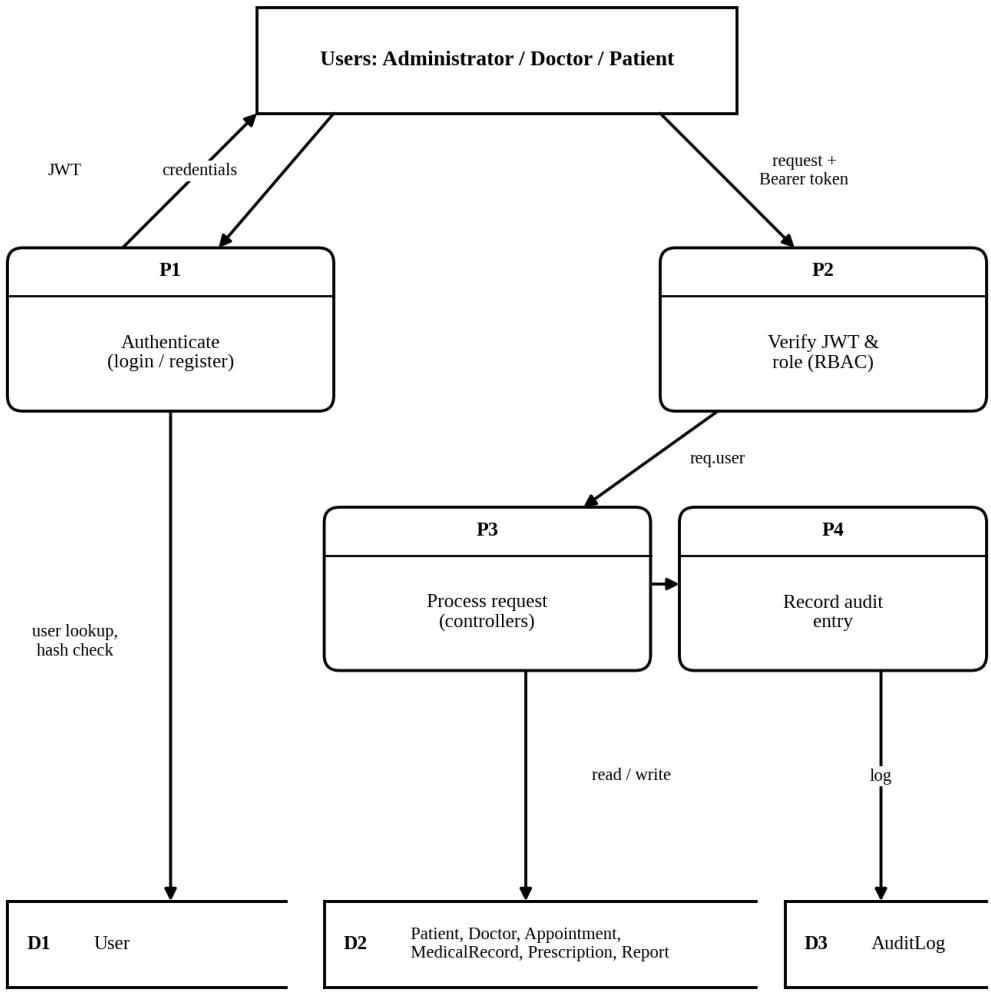


Figure 4.3 Data-Flow Diagram of the EHR System (Level 1) 27

4.4 ENTITY-RELATIONSHIP AND COLLECTIONS SCHEMA

MongoDB is a document database, so relationships are expressed through ObjectId references. Table 4.3 lists the eight Mongoose models, and Figure 4.4 shows how they refer to the User collection.

Table 4.3 Database Collections and Fields 29

Collection	**Fields**	**References / Notes**
User	name (required); email (required, unique, lowercase, pattern-validated); password (required, select:false); role (enum admin, doctor, patient; default patient); createdAt, updatedAt	Pre-save hook hashes the password
Patient	user; dateOfBirth; gender; bloodGroup; phone; address; medicalHistory	user is required and references User
Doctor	user; specialization; department; experienceYears; phone	user is required and references User
Appointment	patient; doctor; appointmentDate; appointmentTime; reason; status (pending, confirmed, completed, cancelled)	patient and doctor reference User; patient, doctor, and appointmentDate are required
MedicalRecord	patient; doctor; symptoms; diagnosis; treatmentPlan; notes; recordDate	patient and doctor reference User and are required
Prescription	patient; doctor; medications [{name, dosage, frequency, duration}]; instructions; prescribedDate	patient and doctor reference User and are required
Report	patient; doctor; reportType; reportName; reportDate	patient and doctor reference User and are required; metadata only
AuditLog	user; userRole; action; resource; resourceId; description; ipAddress; createdAt	user references User

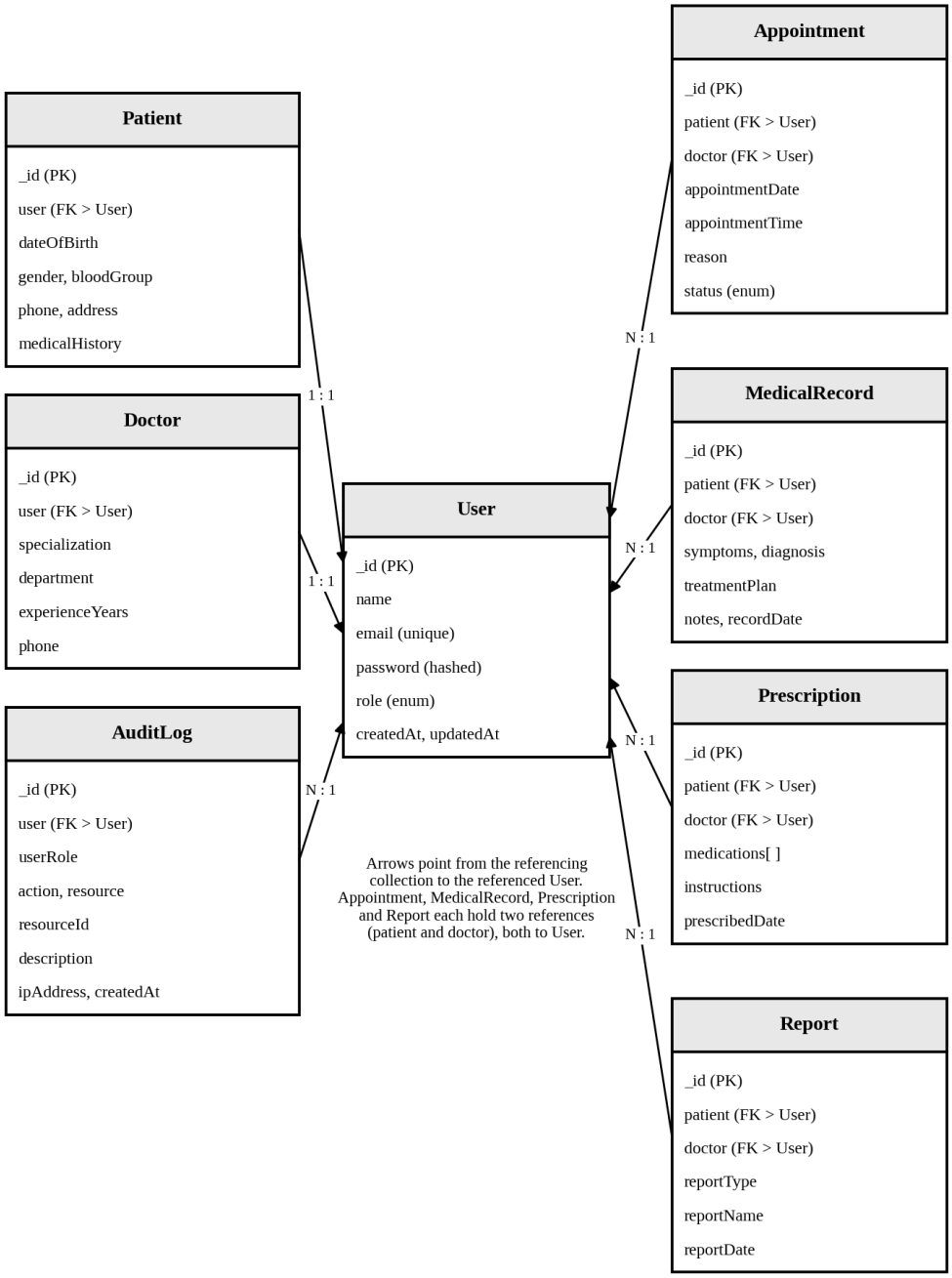


Figure 4.4 Entity-Relationship Diagram of the EHR Collections 29

Controllers normally retrieve data with Model.find(), optionally using populate for related documents, and return

the whole result set. Pagination and filter parameters are not implemented.

4.5 AUTHENTICATION AND AUTHORIZATION SEQUENCES

Figure 4.5Authentication and Authorization Sequence 31

1. **Credential submission:** The login form sends POST /api/auth/login with the email and password.
2. **Database look-up:** The controller finds the user by email and selects the password field.
3. **Password verification:** matchPassword compares the supplied password with the stored bcrypt hash.
4. **Token issue:** generateToken signs a JWT containing the user id, role, and email, and an audit entry is created.
5. **Client storage:** api.saveToken stores the token in localStorage; later requests send it as Authorization: Bearer <token>.
6. **Protected route check:** protect verifies the token and sets req.user; requireRole returns 403 if the role is not allowed; otherwise the controller runs and returns JSON.

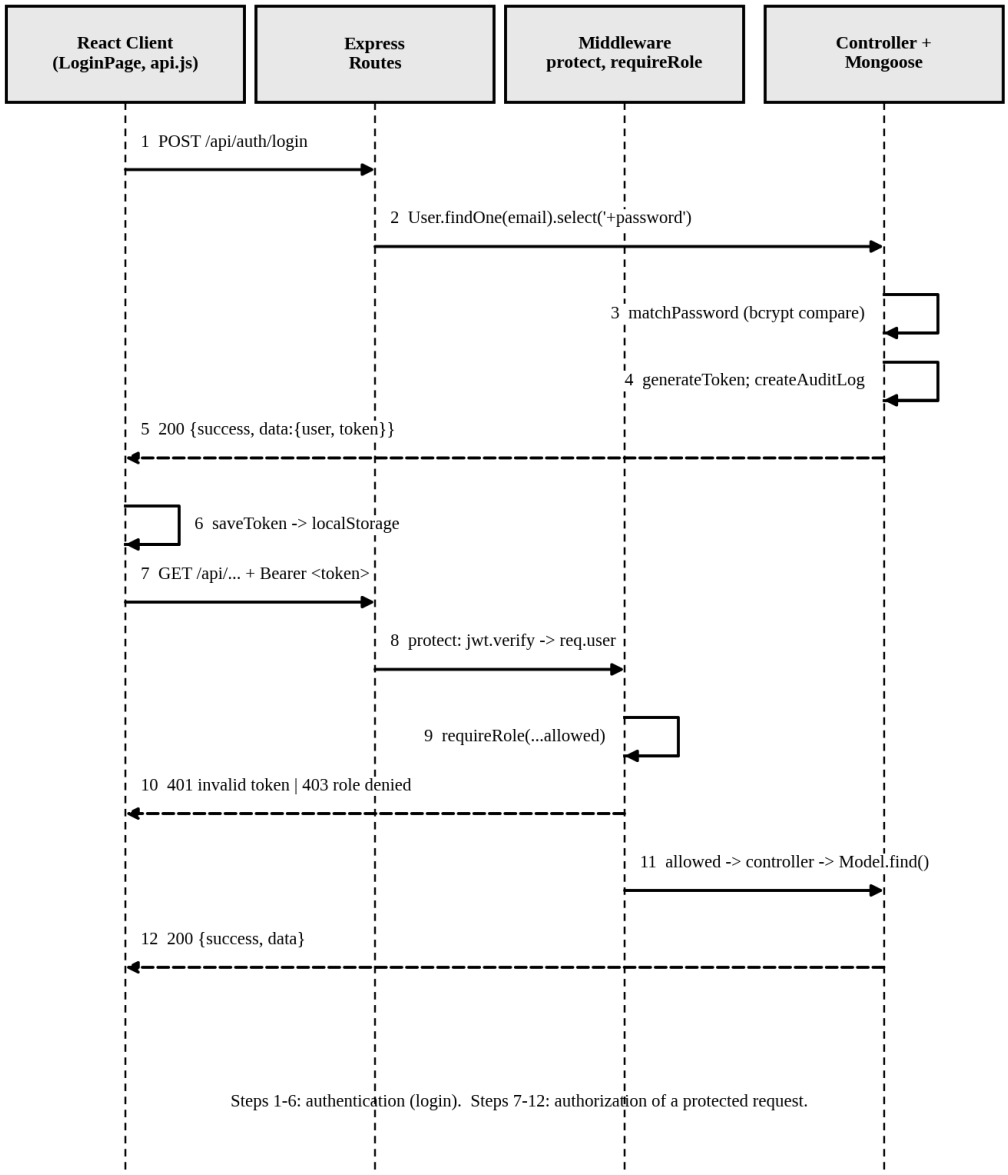


Figure 4.5 Authentication and Authorization Sequence 31

4.6 APPOINTMENT MANAGEMENT WORKFLOW

Figure 4.6Appointment Management Workflow: Backend Path and Current Front-End Behaviour 33

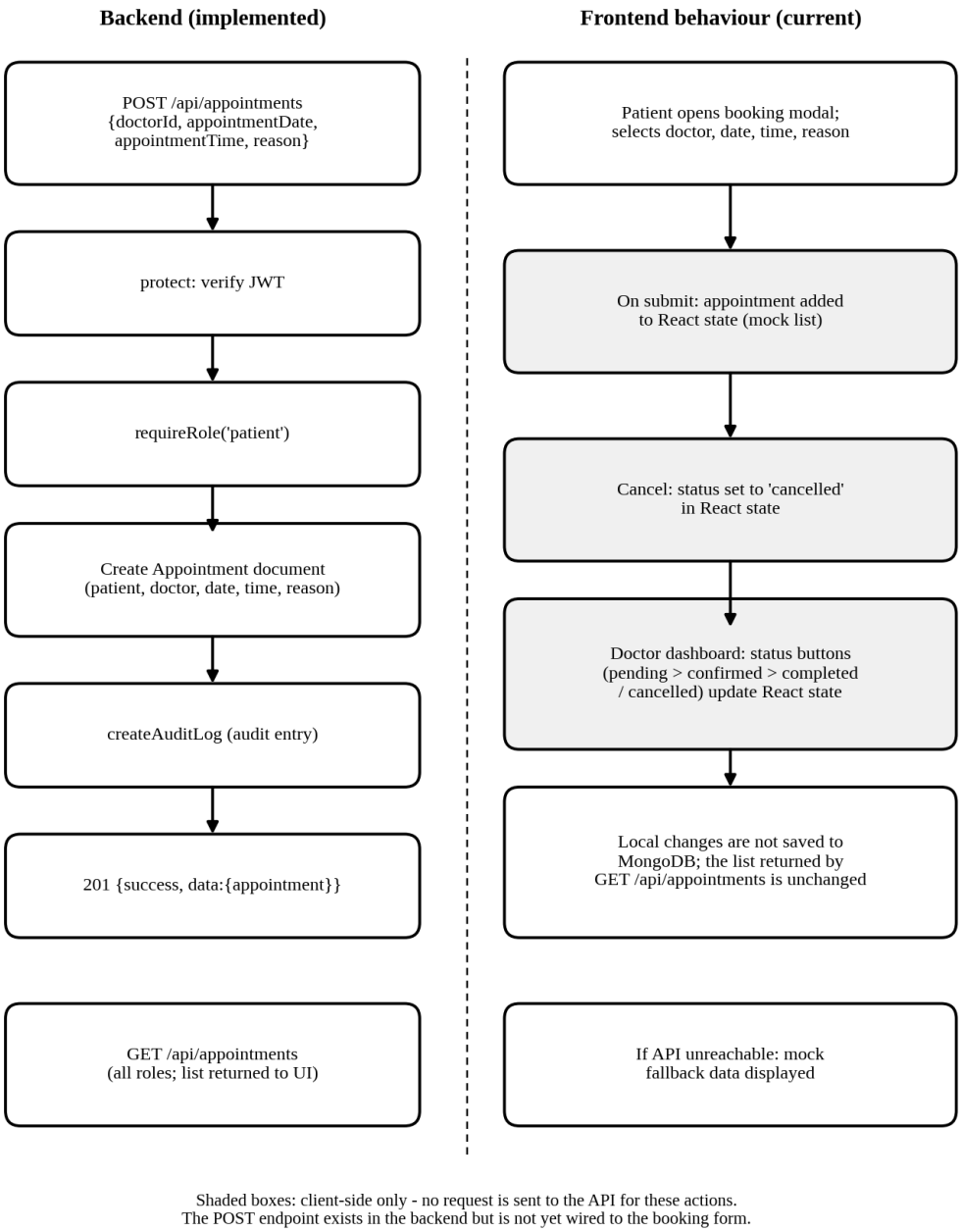


Figure 4.6 Appointment Management Workflow: Backend Path and Current Front-End Behaviour 33

4.7 REST API DESIGN 35

Table 4.4REST API Endpoint Summary 35

Table 4.4 REST API Endpoint Summary 35

Method	Endpoint	Purpose	Auth	Roles	Status
GET	/api/health	Health check	No	-	Implemented
POST	/api/auth/register	Register a user	No	-	Implemented
POST	/api/auth/login	Authenticate and return JWT	No	-	Implemented
GET	/api/auth/me	Current user	Yes	Any	Implemented
GET	/api/users	List users	Yes	Admin	Implemented
GET	/api/patients	List patient profiles	Yes	Admin, Doctor	Implemented
GET	/api/doctors	List doctor profiles	Yes	Admin, Patient	Implemented
GET	/api/appointments	List appointments (role-filtered)	Yes	All	Implemented
POST	/api/appointments	Create appointment	Yes	Patient	Implemented in backend; not used by UI
GET	/api/medical-records	List records	Yes	Doctor, Patient	Implemented
GET	/api/prescriptions	List prescriptions	Yes	Doctor, Patient	Implemented
GET	/api/reports	List report metadata	Yes	Doctor, Patient	Implemented
GET	/api/audit-logs	List audit entries	Yes	Admin	Implemented

The source material also mentions controller functions for creating medical records and updating appointments, but

it does not record routes for them, so they are not listed as endpoints in this report.

CHAPTER 5
RESULTS AND DISCUSSION

This chapter presents the user-interface screens of the implemented application, explains the key implementation mechanisms, reports the testing carried out, and records the implementation status and security posture of the current version. Screenshot placeholders mark where the actual screens are to be inserted.

5.1 USER INTERFACE IMPLEMENTATION & VERIFICATION

The front end is built with React 19 and Vite, using react-router-dom for routing and lucide-react for icons. The pages share DashboardLayout.jsx and use a glass-morphism visual style with cards, badges, and a responsive grid. Each role sees only its own dashboard.

5.1.1 Login Page 38

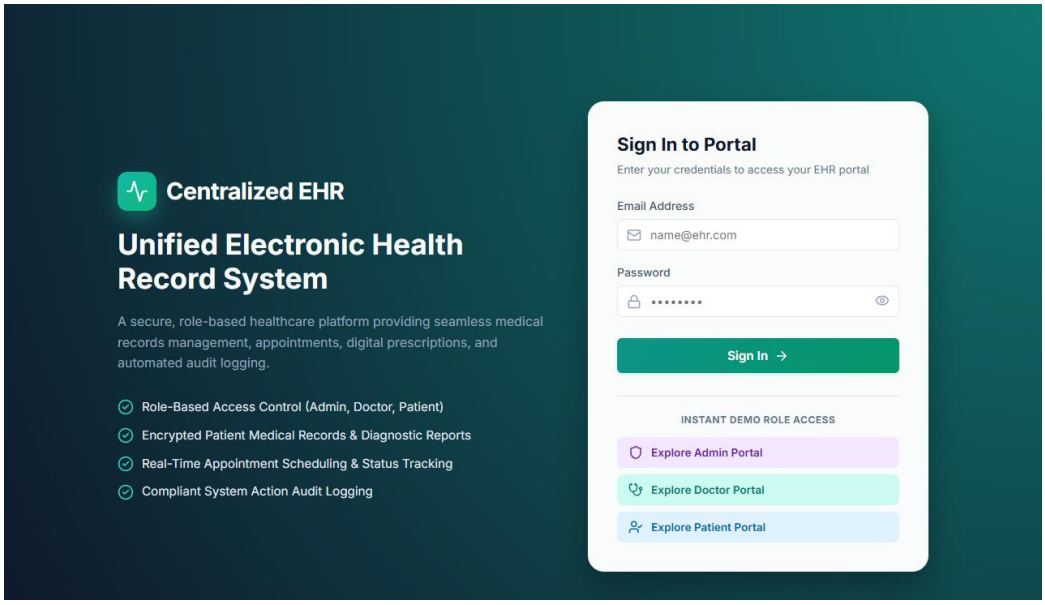


Figure 5.1 Login Page 38

What to show: The login form with the email and password fields. On submit the page calls POST /api/auth/login, stores the token and user, and redirects to the dashboard of the user's role.

5.1.2 Administrator Dashboard 39

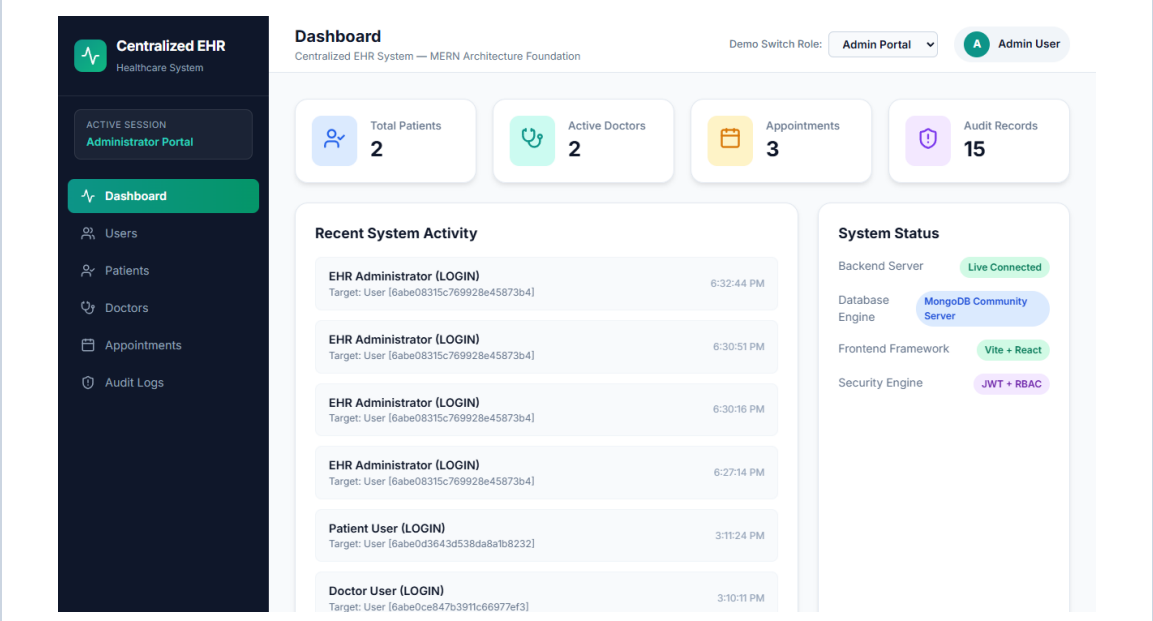


Figure 5.2 Administrator Dashboard Overview 39

What to show: The overview with data fetched from the users, patients, doctors, appointments, and audit-log endpoints, and the system-status badges for the backend connection and MongoDB Community Server.

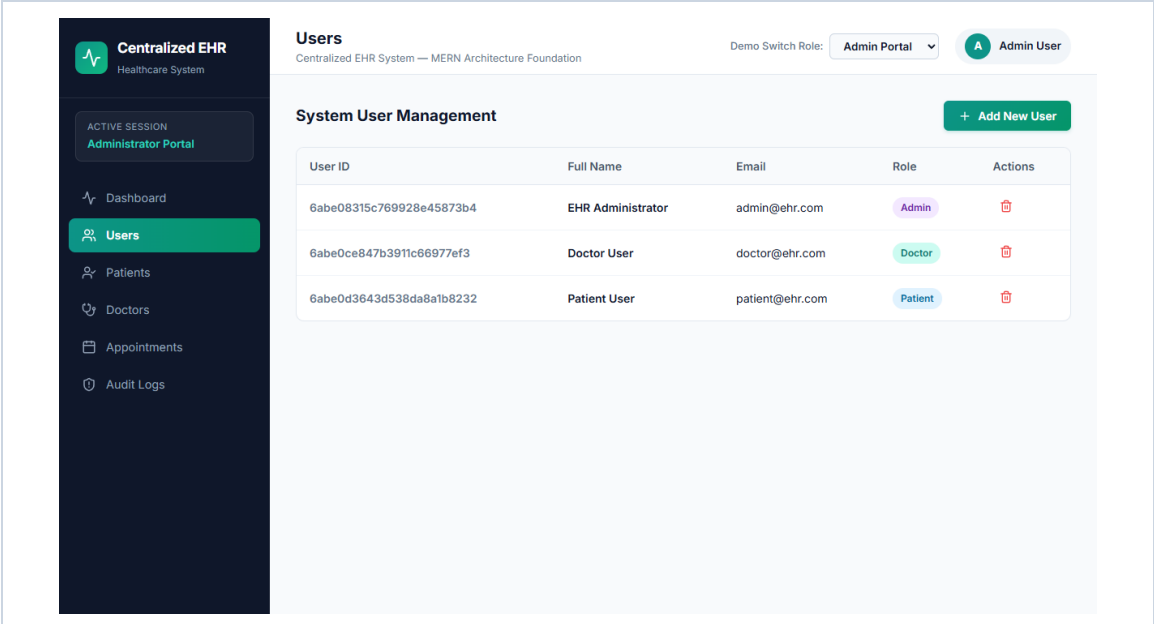


Figure 5.3 Administrator – Users Tab 40

What to show: The table of system users returned by GET /api/users.

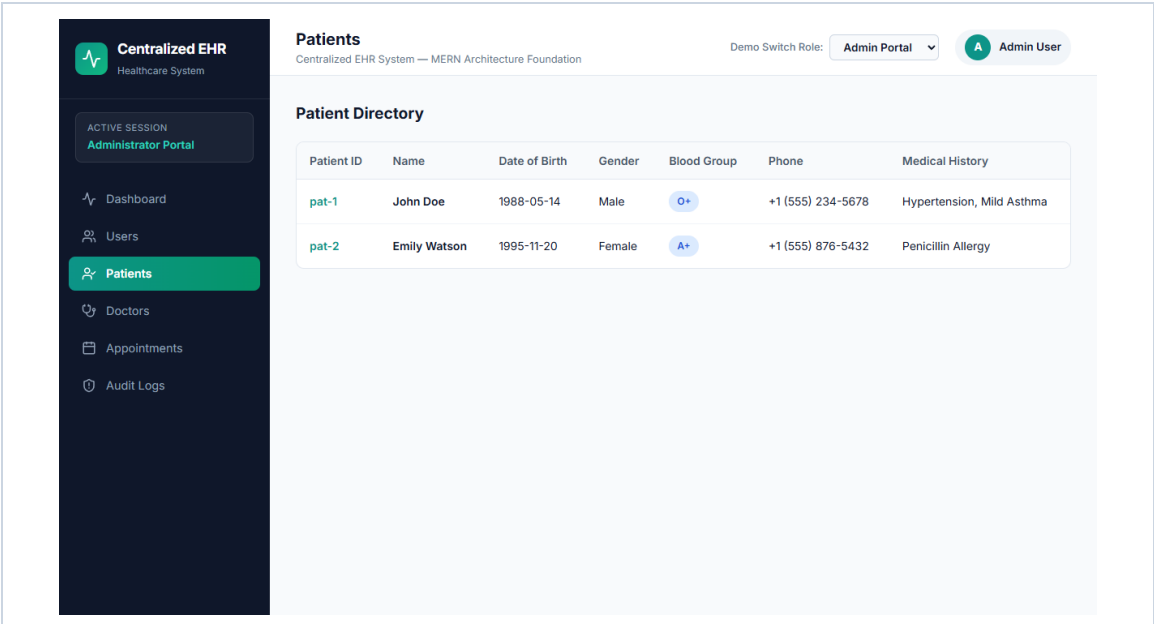


Figure 5.4 Administrator – Patient Management 40

What to show: The Patients tab listing patient profiles returned by GET /api/patients.

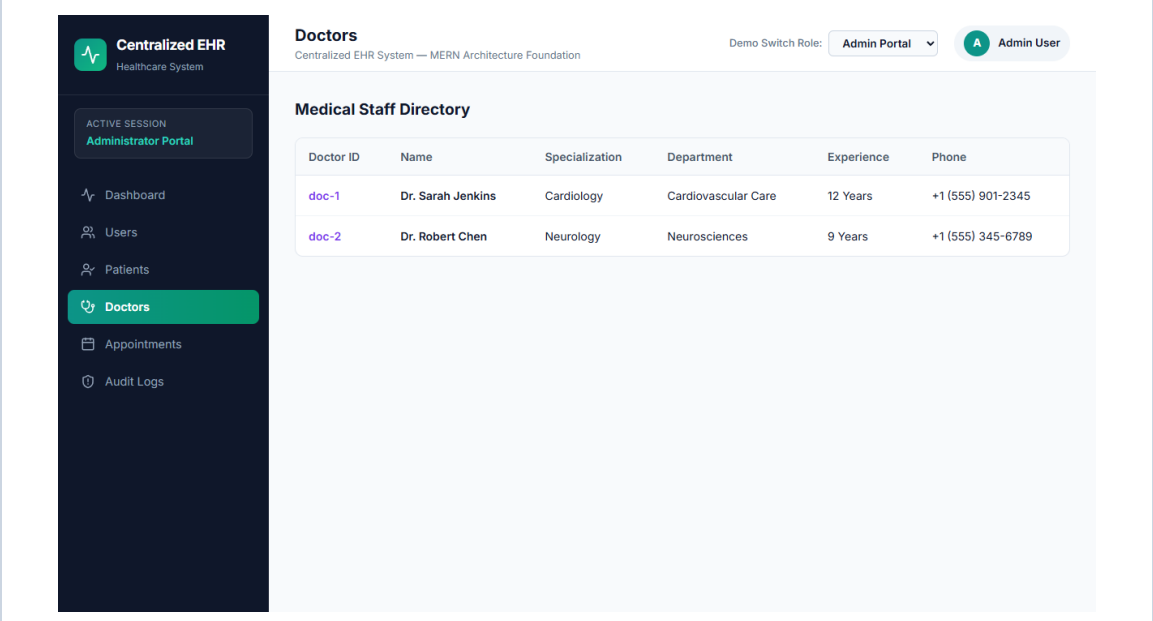


Figure 5.5 Administrator – Doctor Management 41

What to show: The Doctors tab listing doctor profiles returned by GET /api/doctors.

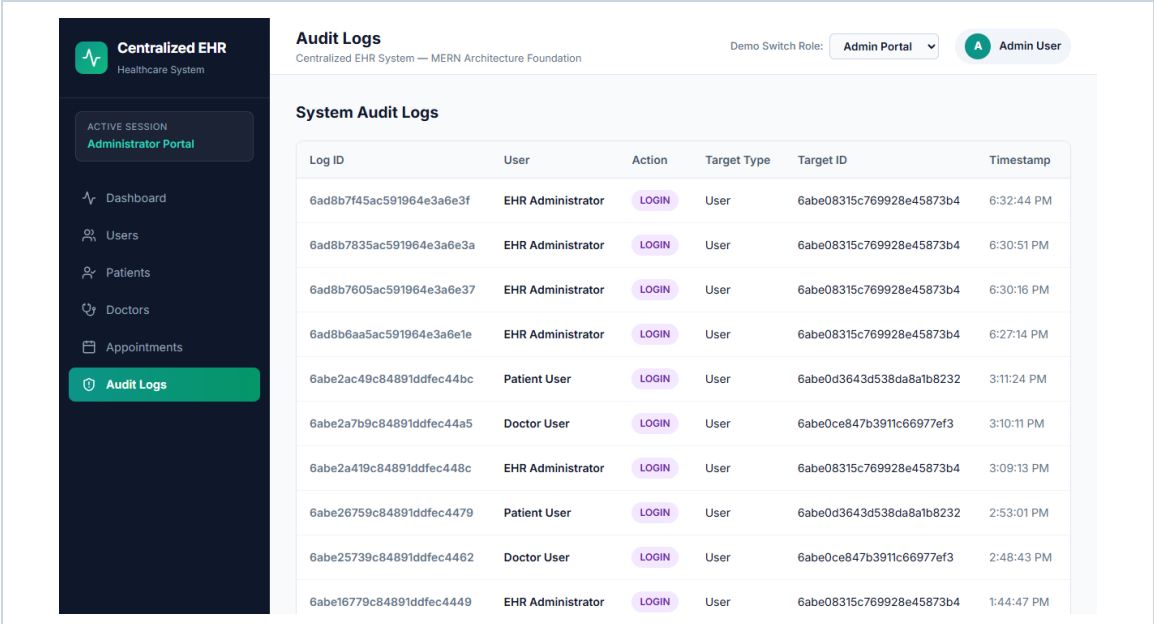


Figure 5.6 Administrator – Audit Logs 42

What to show: The audit-log view (or system-status view that displays the audit-log information) fed by GET /api/audit-logs.

5.1.3 Doctor Dashboard 42

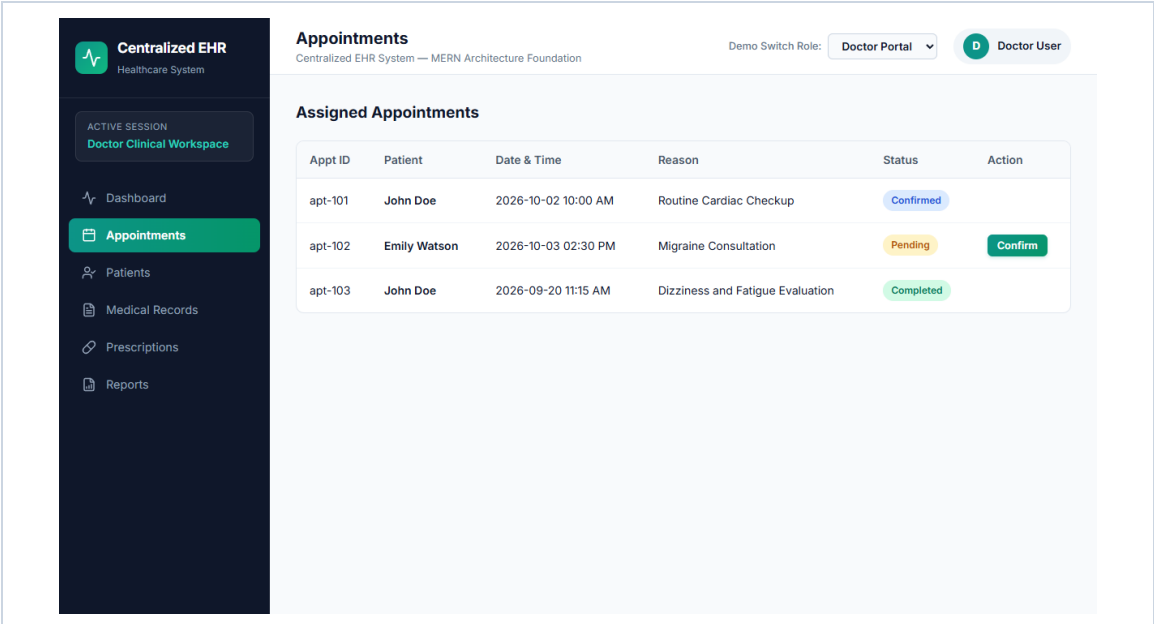


Figure 5.7 Doctor – Appointment Management 43

What to show: The doctor's Appointments tab with the status controls. Status changes shown here are client-side only in the current version.

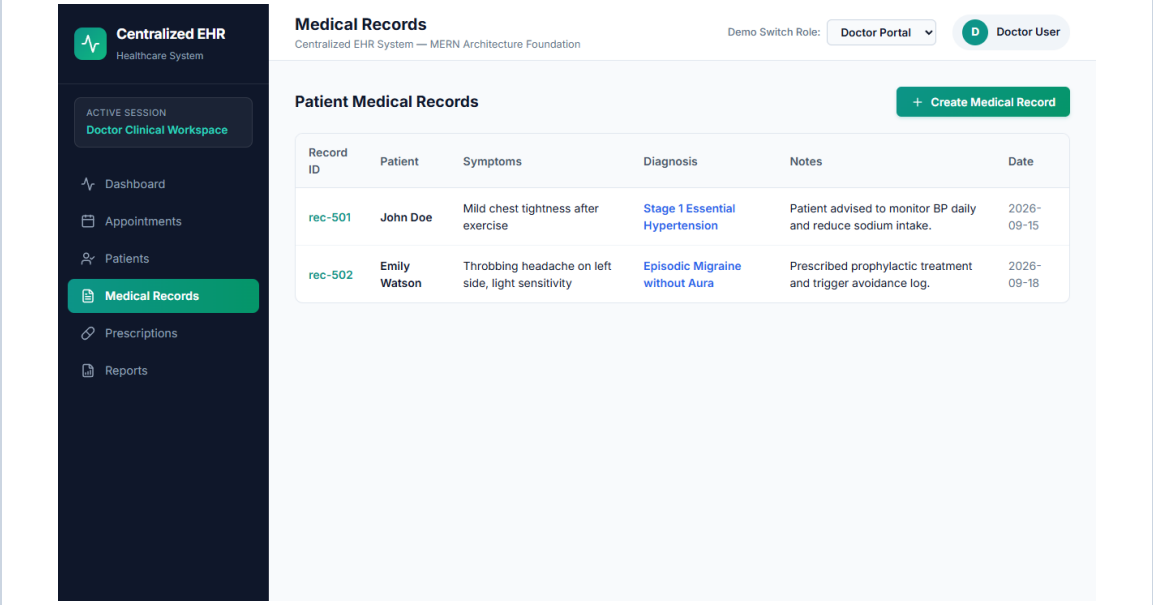


Figure 5.8 Doctor – Medical Records 43

What to show: The Records tab and, if desired, the add-record modal. Records are read from the backend; the modal adds items to client state only.

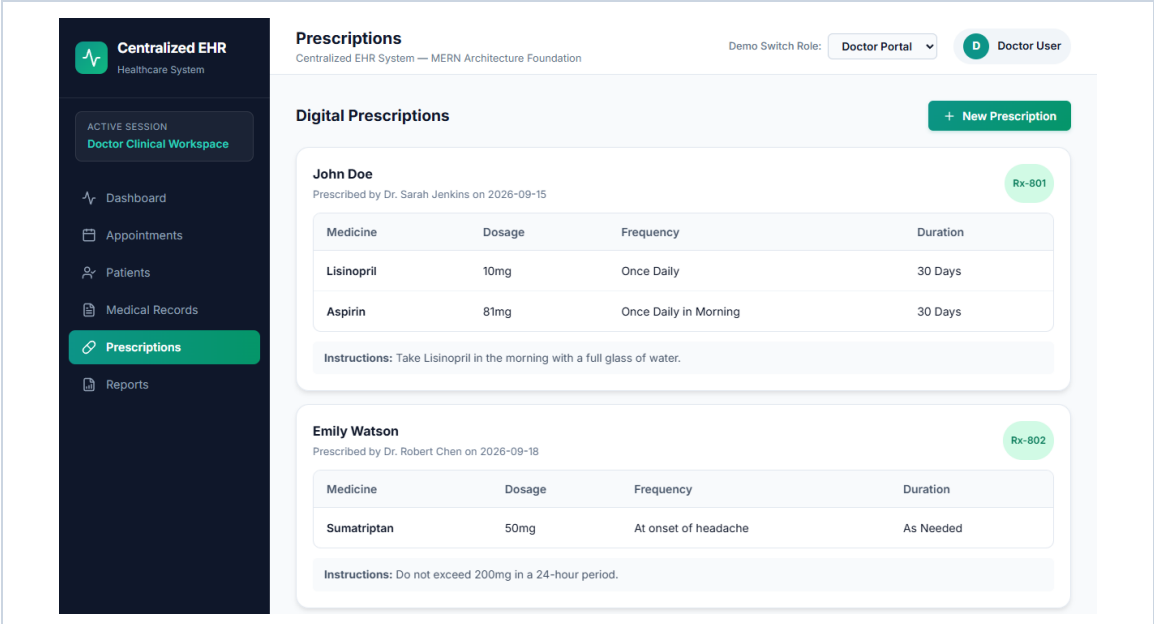


Figure 5.9 Doctor – Prescriptions 44

What to show: The Prescriptions tab and the prescription modal, which updates client state only.

5.1.4 Patient Dashboard 44

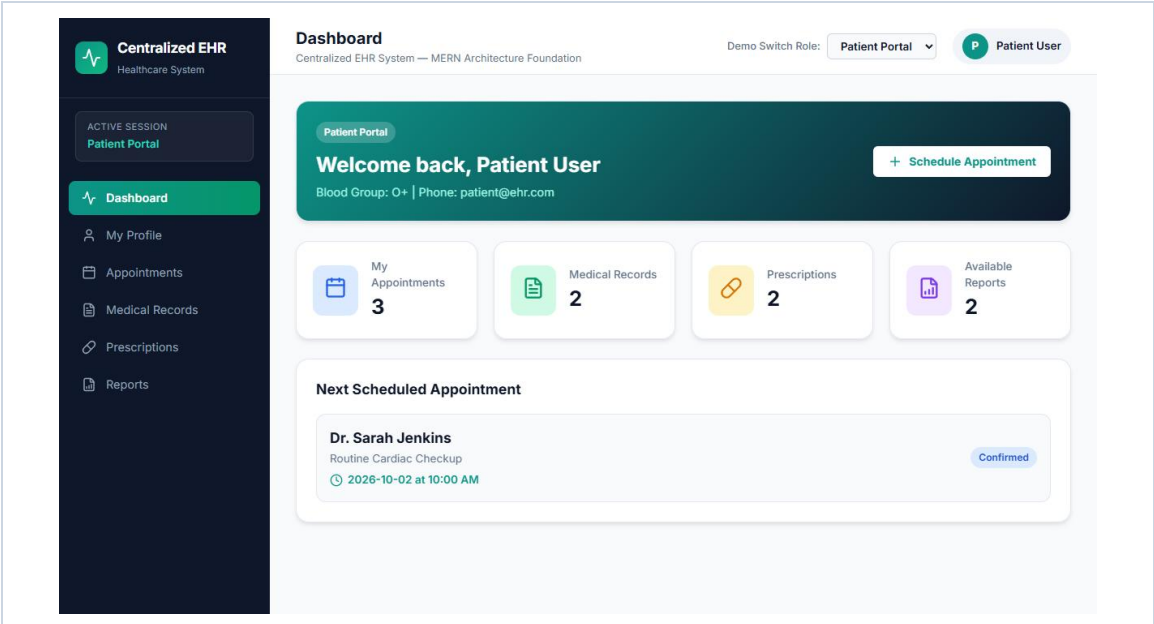


Figure 5.1 Login Page 38

What to show: The patient overview with personal appointments, records, prescriptions, and reports fetched from the backend.

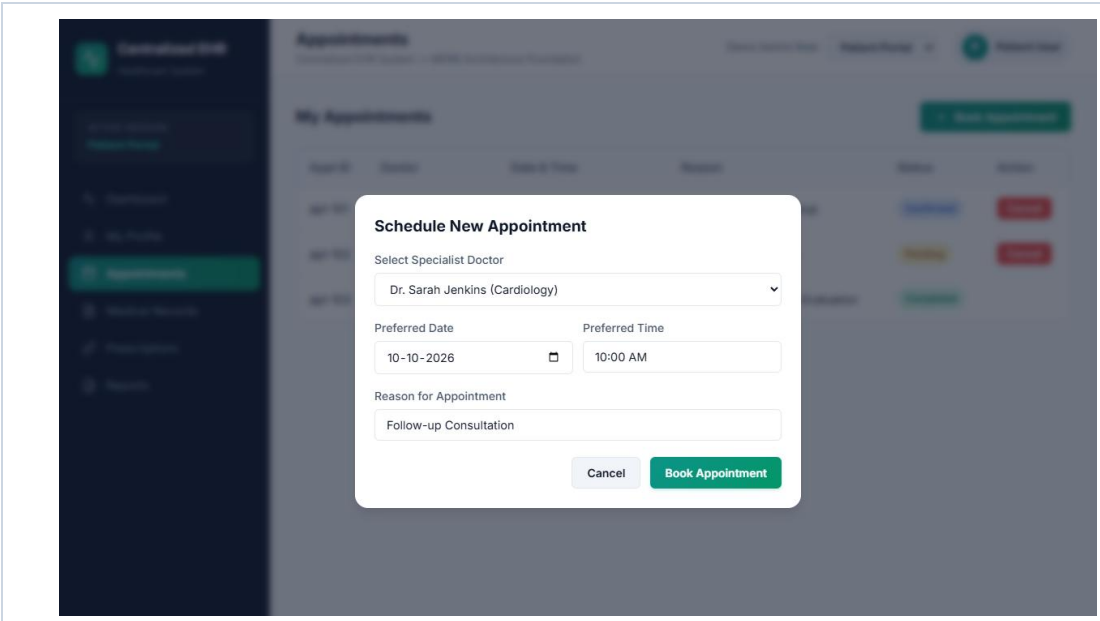


Figure 5.1 Login Page 38

What to show: The booking modal in which the patient selects a doctor, date, time, and reason. Submission updates client state only in the current version.

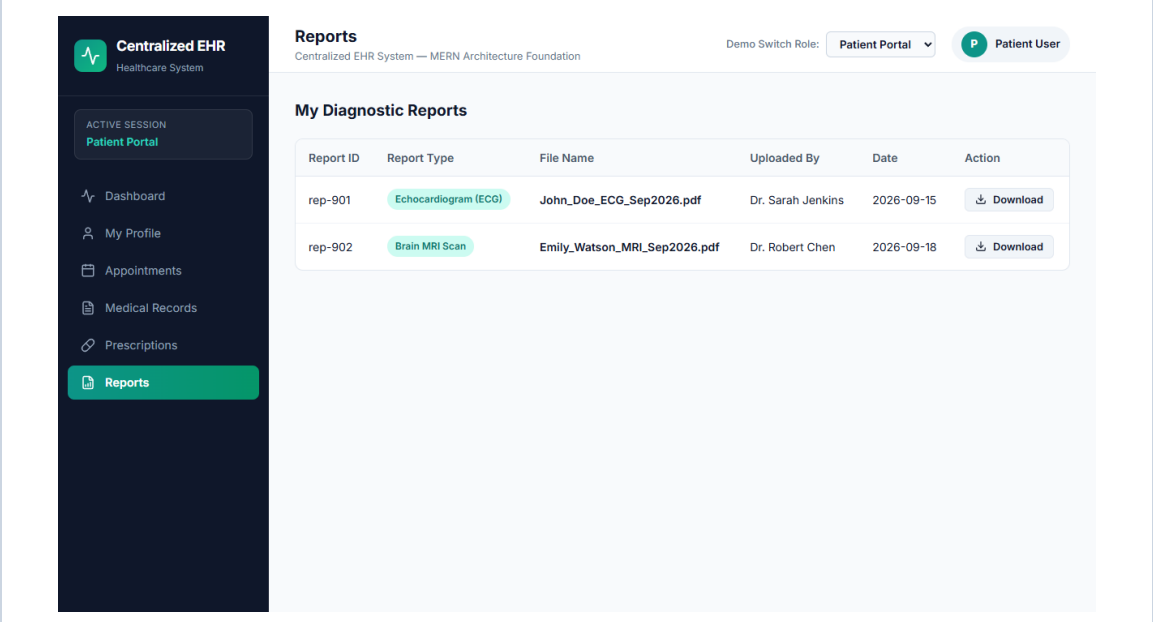


Figure 5.1 Login Page 38

What to show: The Reports tab listing report metadata fetched from GET /api/reports. Upload and download are mocked.

5.2 KEY IMPLEMENTATION DETAILS (CODE EXPLANATION) 46

The following sub-sections explain the central architectural mechanisms of the backend and front end, accompanied by the authoritative source code listings from the project implementation.

5.2.1 MongoDB Connection (backend/config/db.js) 46

The connectDB function reads MONGO_URI from the .env file and connects with mongoose.connect. If the URI is missing or invalid, the process exits, so the server does not start without a database.

```
import mongoose from 'mongoose';


/**
 * Connect to MongoDB Community Server using Mongoose.
 * Exits process with failure code if URI is invalid or connection fails.
 */
export const connectDB = async () => {
  const mongoURI = process.env.MONGO_URI;


  if (
    !mongoURI ||
    mongoURI === 'your_mongodb_connection_string' ||
    mongoURI.includes('<username>')
  ) {
    console.error('❌ MONGODB ERROR: Valid MONGO_URI is missing in backend/.env file.');
    process.exit(1);
  }


  try {
    const conn = await mongoose.connect(mongoURI);
    console.log(`☑ MongoDB Connected successfully: ${conn.connection.host}`);
  } catch (error) {
    console.error(`❌ MongoDB Connection Error: ${error.message}`);
    process.exit(1);
  }
};


export default connectDB;
```

Listing 5.1: Database Connection Implementation (backend/config/db.js)

5.2.2 Password Hashing (backend/models/User.js) 47

The User schema defines a pre-save hook that generates a salt with bcrypt.genSalt(10) and hashes the password with bcrypt.hash. The password field is declared with select:false, so it is excluded from query results unless a query explicitly selects it, as the login controller does. A matchPassword method compares a candidate password with the stored hash.

```
import mongoose from 'mongoose';
import bcrypt from 'bcryptjs';


const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: [true, 'Please provide a name'], trim: true },
    email: {
      type: String,
      required: [true, 'Please provide an email'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/, 'Please provide a valid
email']
    },
    password: { type: String, required: [true, 'Please provide a password'], minlength:
6, select: false },
    role: { type: String, enum: ['admin', 'doctor', 'patient'], default: 'patient' }
  },
  { timestamps: true }
);


// Pre-save hook: automatically salt and hash password before saving to MongoDB
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});


// Compare entered candidate password with stored bcrypt hash
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};


// Safe JSON serialization excluding password and internal version keys
userSchema.methods.toSafeObject = function () {
  const obj = this.toObject ? this.toObject() : { ...this };
  delete obj.password;
  delete obj.__v;
  return obj;
};


const User = mongoose.model('User', userSchema);
export default User;
```

Listing 5.2: User Model & Bcrypt Pre-Save Hashing (backend/models/User.js)

generateToken in auth.controller.js signs a payload containing the user id, role, and email using JWT_SECRET, with the expiry taken from JWT_EXPIRE (default 30 days). In auth.middleware.js, protect reads the Authorization header, verifies the token, and attaches the decoded payload to req.user. The source notes that a fallback default secret exists if JWT_SECRET is not set; this should be removed for any real deployment (see Section 5.5).

```
import jwt from 'jsonwebtoken';
import User from '../models/User.js';


// JWT Minting: Signs user identity, role, and email into HMAC-SHA256 bearer token
export const generateToken = (id, role, email) => {
  const secret = process.env.JWT_SECRET || 'supersecret_ehr_jwt_key_2026';
  const expiresIn = process.env.JWT_EXPIRE || '30d';
  return jwt.sign({ id, role, email }, secret, { expiresIn });
};


// JWT Verification Guard: Intercepts Authorization Bearer token on protected routes
export const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }


  if (!token) {
    return res.status(401).json({ success: false, message: 'Not authorized. Missing
token.' });
  }


  try {
    const secret = process.env.JWT_SECRET || 'supersecret_ehr_jwt_key_2026';
    const decoded = jwt.verify(token, secret);


    // Retrieve fresh user profile omitting password hash
    const user = await User.findById(decoded.id).select('-password');
    if (!user) {
      return res.status(401).json({ success: false, message: 'User account no longer
exists.' });
    }
    req.user = user;
    next();
  } catch (error) {
    return res.status(401).json({ success: false, message: 'Not authorized. Invalid or
expired token.' });
  }
};
```

Listing 5.3: Stateless JWT Generation and Verification (backend/middleware/auth.middleware.js)

5.2.4 Role-Based Authorization Middleware 50

requireRole(...roles) in role.middleware.js returns a middleware function that checks whether req.user.role is one

of the allowed roles and otherwise responds with HTTP 403. Routes combine protect and requireRole; for example, /api/users is restricted to administrators.

```
/**
 * Role-Based Access Control (RBAC) Middleware
 * Restricts endpoint execution to authorized user roles (admin, doctor, patient)
 */
export const requireRole = (...roles) => {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({
        success: false,
        message: 'Authentication required before checking role permissions.'
      });
    }


    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: `Forbidden: User role '${req.user.role}' is not authorized to access
this route.`
      });
    }


    next();
  };
};


export const authorize = requireRole;
```

Listing 5.4: Role-Based Authorization Guard Middleware (backend/middleware/role.middleware.js)

5.2.5 Front-End API Service and Route Guards 51

services/api.js reads the token from localStorage, adds the Authorization header, sets the JSON content type, and returns { success, data, message, status }. In App.jsx, ProtectedRoute checks the stored token and user and the

required role, and PublicRoute redirects logged-in users away from the login page to their own dashboard. Dashboard pages catch API errors, log a console warning, and fall back to mock data.

```
// frontend/src/services/api.js - Central HTTP Client with Bearer token injection
const API_BASE_URL = 'http://localhost:5000/api';
export const getToken = () => localStorage.getItem('ehr_token');
export const getUser = () => JSON.parse(localStorage.getItem('ehr_user') || 'null');


const request = async (endpoint, options = {}) => {
  const token = getToken();
  const headers = { 'Content-Type': 'application/json', ...(options.headers || {}) };
  if (token) headers['Authorization'] = `Bearer ${token}`;


  const response = await fetch(`${API_BASE_URL}${endpoint}`, { ...options, headers });
  const data = await response.json().catch(() => ({}));
  if (!response.ok) {
    const error = new Error(data.message || `HTTP ${response.status}`);
    error.status = response.status;
    throw error;
  }
  return data;
};


// frontend/src/App.jsx - React Route Guard enforcing role-specific navigation
function ProtectedRoute({ allowedRole, children }) {
  const token = getToken();
  const user = getUser();
  if (!token || !user) return <Navigate to="/login" replace />;


  const userRole = (user.role || '').toLowerCase();
  if (allowedRole && userRole !== allowedRole.toLowerCase()) {
    if (userRole === 'admin') return <Navigate to="/admin" replace />;
    if (userRole === 'doctor') return <Navigate to="/doctor" replace />;
    return <Navigate to="/patient" replace />;
  }
  return children;
}
```

Listing 5.5: Front-End API Client and ProtectedRoute Guard (frontend/src/services/api.js, App.jsx)

createAuditLog in services/auditLog.service.js receives the details of an action (user, role, action type, resource, description, and IP address) and stores them as an AuditLog document. It is called after successful registration and login and by data-modifying controllers.

```
import AuditLog from '../models/AuditLog.js';


/**
 * Audit Logging Service
 * Safely persists immutable audit trail records in MongoDB for authentication and data
modifications
 */
export const createAuditLog = async ({
  user = null,
  userRole = 'guest',
  action,
  resource,
  resourceId = '',
  description = '',
  ipAddress = ''
}) => {
  try {
    // Only attempt database insert if MongoDB connection is active
    if (AuditLog.db && AuditLog.db.readyState === 1) {
      await AuditLog.create({
        user: user || null,
        userRole,
        action,
        resource,
        resourceId,
        description,
        ipAddress
      });
    }
  } catch (error) {
    // Non-blocking log to console to protect primary API transaction integrity
    console.error('[AUDIT LOG SERVICE ERROR]:', error.message);
  }
};
```

Listing 5.6: Asynchronous Audit Logging Service (backend/services/auditLog.service.js)

5.3 TESTING AND VALIDATION 54

5.3.1 Testing Methodology 54

The backend was exercised with Node.js test scripts in the backend/tests folder, including a general unit test, seven phase-wise functional tests (phase2a-test.js to phase2g-test.js), and a health check. The mongodb-memory-server package is available for tests that need a temporary database. The frontend was verified by a production build with Vite. Seed scripts were run to create demonstration accounts and verify login. Table 5.1 summarises the evidence available.

Table 5.1 Testing Summary 54

Test / Activity	**Scope**	**Recorded Result**
tests/unit-test.js	General unit tests for models and services	Executed; exit code 0
tests/phase2a-test.js to phase2g-test.js	Functional tests of the Phase-2 features (users, patients, doctors, appointments, records, prescriptions, reports)	Executed; exit code 0
tests/check-health.js	Health endpoint	Executed; passed
npm run build (frontend)	Compilation and bundling	Completed without errors
createDemoAdmin.js, setupDoctorAccount.js, setupPatientAccount.js	Demo accounts exist and login works	Ran successfully; doctor and patient logins verified

The recorded evidence consists of process exit codes and completion messages. Individual test-case counts, assertions, and response times were not recorded, so none are reported here. If the console output of the test runs is available, it can be inserted below.

[INSERT ACTUAL TEST-RUN CONSOLE OUTPUT SCREENSHOT HERE (optional)]

5.3.2 Untested Areas **55**

There are no UI component tests, no end-to-end browser tests, no form-validation tests, no file-upload tests (the feature does not exist), no pagination tests, and no security-hardening tests such as rate limiting or CSRF. The application has not been clinically validated or load tested.

5.4 IMPLEMENTATION STATUS **55**

Table 5.2Feature Implementation Status

55

Table 5.2 Feature Implementation Status 55

Feature	**Status**	**Notes**
Registration, login, JWT issue	Implemented	Passwords hashed with bcryptjs; token returned on success
Server-side RBAC	Implemented	protect and requireRole on protected routes
Client-side route guarding	Implemented	ProtectedRoute and PublicRoute
Read endpoints for all entities	Implemented	Role-restricted GET routes; no pagination
Admin dashboard data	Implemented	Fetched from backend; mock fallback on failure
Doctor and patient dashboards data	Implemented	Fetched from backend; mock fallback on failure
Audit logging and viewing	Implemented	Basic entries; no before/after snapshots
Create appointment (API)	Implemented in backend	Endpoint exists; not wired to the booking form
Book / cancel appointment in UI	Client-side only	Updates React state; not persisted
Doctor changes appointment status in UI	Client-side only	Updates React state; not persisted
Create medical record in UI	Client-side only	Modal adds to React state
Create prescription in UI	Client-side only	Modal adds to React state
Report upload / download	Not implemented	Only metadata is stored; interface is mocked
Email verification, password reset	Not implemented	-
Rate limiting, brute-force protection	Not implemented	-
Pagination, advanced search	Not implemented	GET endpoints return full collections

5.5 SECURITY DISCUSSION 57

Table 5.3Security Measures: Implemented and Not Implemented 57

Table 5.3 Security Measures: Implemented and Not Implemented 57

Area	**Implemented**	**Not implemented / recommended**
Password storage	bcryptjs hash with 10 salt rounds; password excluded from queries by default	Password-strength policy and reset flow
Authentication	JWT verified on protected routes; expiry from JWT_EXPIRE (default 30 days)	Multi-factor authentication; shorter token lifetime and refresh strategy
Authorization	RBAC on server routes and client routes	Finer-grained permissions; per-record ownership rules beyond what is coded
Transport	CORS restricted to the configured client origin	HTTPS enforcement (depends on deployment)
Abuse protection	None	Rate limiting and brute-force protection
Data protection	Passwords hashed	Encryption of other stored data
Secrets handling	Secrets read from environment variables	Remove the fallback default JWT secret; manage secrets outside source control
Auditability	Audit entries for authentication and data-modifying actions	Before/after change records; tamper-resistant storage
Token storage on client	Token kept in localStorage	Consider alternatives that reduce exposure to script injection

The results show a working end-to-end MERN skeleton: a user can log in, receive a role-bearing JWT, and see a dashboard populated from the backend, while the server rejects requests that lack a valid token or the required role. The principal gap is that several creation workflows exist only as interface behaviour. Closing this gap requires wiring the existing forms to the corresponding POST and PUT endpoints and adding the missing routes, which is the first item of the future work in Chapter 6.

CHAPTER 6
CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION **59**

The Centralized Electronic Health Record (EHR) System demonstrates a MERN application that stores users, patient and doctor profiles, appointments, medical records, prescriptions, report metadata, and audit logs in one MongoDB database behind a REST API. It authenticates users with JWT, hashes passwords with bcryptjs, enforces role-based access on both server and client, and presents separate dashboards for administrators, doctors, and patients.

Key accomplishments of the current version are:

- **Centralised data model:** Eight Mongoose models with references to the User collection.
- **Security baseline:** bcrypt password hashing, JWT authentication, and RBAC middleware.

- **Audit trail:** Audit entries for authentication and data-modifying actions, viewable by administrators.
- **Role-specific user interface:** Administrator, doctor, and patient dashboards in a shared layout that read live data from the backend.
- **Verification:** Backend test scripts that ran with exit code 0 and a successful frontend production build.

The project also has clear limitations. Creation of medical records, prescriptions, and report entries and the booking of appointments from the interface are not yet persisted; reports have no file upload; and security hardening, pagination, input validation, and broader test coverage remain to be done. The system is a sound academic foundation, not a production or clinically validated system.

6.2 FUTURE ENHANCEMENTS 60

The following enhancements are proposed. None of them is implemented in the current version.

1. **Persist user-interface actions:** Add or complete POST and PUT endpoints for medical records, prescriptions,

reports, and appointment updates, and connect the existing forms to them with api.post and api.put.

2. **File uploads for reports:** Add a multipart upload endpoint, store files in GridFS or cloud storage, and provide download links.
3. **Security hardening:** Add rate limiting (for example with express-rate-limit), enforce TLS, add email-based password reset and email verification, consider two-factor authentication, and remove the fallback default JWT secret.
4. **Input validation:** Use a validation library such as Joi or express-validator to sanitise requests before controller logic.
5. **Pagination and filtering:** Add limit and skip parameters and filters such as date and status.
6. **Wider testing:** Add Jest and React Testing Library tests for the interface and Cypress tests for end-to-end flows.
7. **Finer-grained roles:** Introduce a permission matrix or additional roles such as nurse or laboratory technician.

8. **Richer audit logs:** Record changed fields and store entries in tamper-resistant storage.
9. **Deployment:** Containerise with Docker, add environment-specific configuration, and set up a CI/CD pipeline.
10. **Accessibility:** Conduct an accessibility audit and add ARIA labels, keyboard navigation, and improved contrast.

SOURCE CODE & TECHNICAL ARTIFACTS

A.1 PROJECT DIRECTORY STRUCTURE 63

The key folders and files of the project are shown below.

```
Centralized EHR/
├─ backend/
│   ├─ .env                        (not committed; values omitted)
│   ├─ server.js                   entry point
│   ├─ app.js                      Express app, middleware, routes
│   ├─ config/db.js                Mongoose connection
│   ├─ middleware/
│   │    ├─ auth.middleware.js     JWT verification (protect)
│   │    ├─ role.middleware.js     RBAC guard (requireRole)
│   │    └─ error.middleware.js    404 and error handling
│   ├─ models/                     User, Patient, Doctor, Appointment,
│   │                              MedicalRecord, Prescription, Report,
│   │                              AuditLog
│   ├─ controllers/                auth, user, patient, doctor,
│   │                              appointment, medicalRecord,
│   │                              prescription, report, auditLog
│   ├─ routes/                     *.routes.js for each controller
│   ├─ services/auditLog.service.js
│   ├─ scripts/                    createDemoAdmin.js,
│   │                              setupDoctorAccount.js,
│   │                              setupPatientAccount.js
│   └─ tests/                      unit-test.js, phase2a..2g-test.js,
│                                  check-health.js
└─ frontend/
    ├─ package.json, vite.config.js
    └─ src/
       ├─ App.jsx                  routes, ProtectedRoute, PublicRoute
       ├─ services/api.js          fetch wrapper and token helpers
       ├─ data/mockData.js         fallback arrays
       ├─ layouts/DashboardLayout.jsx
       └─ pages/                   LoginPage, AdminDashboard,
                                   DoctorDashboard, PatientDashboard
```

A.2 ENVIRONMENT CONFIGURATION 64

The backend reads its configuration from environment variables in backend/.env. Only the variable names are listed; real values, especially the database connection string and the JWT secret, must never be included in a report or committed to source control.

Table A.1 Backend Environment Variables 64

Variable	**Purpose**
MONGO_URI	MongoDB connection string used by config/db.js (value omitted)
JWT_SECRET	Secret used to sign and verify JWTs (value omitted)
JWT_EXPIRE	Token lifetime; default 30d if not set
CLIENT_URL	Allowed CORS origin; defaults to http://localhost:5173
PORT	Server port; defaults to 5000

A.3 SAMPLE REST API REQUESTS AND RESPONSES 64

The examples below show the documented request fields and response structure. Values in angle brackets are placeholders and do not represent real data.

Example 1: Log in and receive a JWT

```
POST /api/auth/login
Content-Type: application/json


{ "email": "<email>", "password": "<password>" }


200 OK
{ "success": true,
  "data": { "user": { ... }, "token": "<jwt>" } }
```

Example 2: Create an appointment as a patient

```
POST /api/appointments
Authorization: Bearer <jwt>
Content-Type: application/json


{ "doctorId": "<doctor user id>",
  "appointmentDate": "<date>",
  "appointmentTime": "<time>",
  "reason": "<reason>" }


200/201 { "success": true,
  "data": { "appointment": { ... } } }
```

The exact success status code for the second example was not recorded in the source material and should be confirmed from the controller.

A.4 SEED SCRIPTS **65**

Table A.2 Seed and Setup Scripts 65

Script	**Purpose**
scripts/createDemoAdmin.js	Creates a demonstration administrator account
scripts/setupDoctorAccount.js	Creates a demonstration doctor account and verifies login
scripts/setupPatientAccount.js	Creates a demonstration patient account and verifies login

1. Häyrinen, K., Saranto, K., & Nykänen, P. (2008). Definition, structure, content, use and impacts of electronic health records: A review of the research literature. International Journal of Medical Informatics, 77(5), 291–303.
2. Institute of Medicine. (2003). Key Capabilities of an Electronic Health Record System: Letter Report. Washington, DC: The National Academies Press.
3. Ferraiolo, D., & Kuhn, R. (1992). Role-Based Access Controls. Proceedings of the 15th National Computer Security Conference, pp. 554–563.
4. Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-Based Access Control Models. IEEE Computer, 29(2), 38–47.
5. Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). IETF, RFC 7519. https://datatracker.ietf.org/doc/html/rfc7519
6. Provos, N., & Mazières, D. (1999). A Future-Adaptable Password Scheme. Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference, pp. 81–91.
7. OWASP Foundation. (2021). OWASP Top 10:2021. https://owasp.org/Top10/
8. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.

9. MongoDB Documentation. MongoDB Inc. https://www.mongodb.com/docs/
10. Express.js Documentation. OpenJS Foundation. https://expressjs.com/
11. Mongoose Documentation. https://mongoosejs.com/
12. React Documentation. https://react.dev/
13. Node.js Documentation. OpenJS Foundation. https://nodejs.org/
14. Vite Documentation. https://vite.dev/guide/